Available online at www.sciencedirect.com

ScienceDirect

Procedia CIRP 134 (2025) 735–740

www.elsevier.com/locate/procedia

Proceedings of the 58th CIRP Conference on Manufacturing Systems 2025

Edge-Deployed Deep Learning for Automated Quality Control in Industrial Assembly: A Case Study in Real-Time Defect Detection

Milad Ashourpour[a*], Ghazaleh Azizpour[a,b]

[a]*Jönköping School of Engineering, Jönköping, 55111, Sweden*
[b]*Husqvarna AB, Huskvarna, 56182, Sweden*

* Corresponding author. Tel.: +46 36 550 2412. *E-mail address:* milad.ashourpour@ju.se

Abstract

Recent advancements in computer vision (CV) and deep learning (DL) have significantly enhanced automated quality control in manufacturing. The use of deep Convolutional Neural Networks (CNNs) has revolutionized inspections by offering fast, accurate, and cost-effective solutions that minimize human intervention. This study focuses on a visual inspection task for quality control (QC) on an assembly line that is powered by a YOLOv8-based DL algorithm and integrates a robotic process control on the line. Through experimental evaluation, two YOLOv8 are trained and evaluated on a dataset containing annotated images collected from an active assembly line: an optimized YOLOv8.2s model and a verification YOLOv8.2m model. The optimized model achieves mAP50 of 0.972 and mAP50-95 of 0.647 across all classes, while the verification model shows marginal improvement with mAP50 of 0.979 and mAP50-95 of 0.673. However, this comes at the cost of noticeable increased computational demands, with training time increasing from 14 hours to 46 hours. The findings demonstrate that while larger models may offer slight performance improvements (2.6-3.8% in mAP50-95), the trade-off between computational resources and performance gains must be carefully considered in industrial applications.

© 2025 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the International Programme committee of the 58th CIRP Conference on Manufacturing Systems

Keywords: Robotic Automation; Deep Learning; Defect Detection, YOLOv8; Assembly Line

1. Introduction

Quality control (QC) is a crucial aspect of operations in assembly lines, with the overall objective of ensuring that products meet specified standards and conform to safety regulations. The overall productivity of assembly lines could be directly linked to the ability to maintain quality and identify defects consistently. Common quality problems in assembly lines include defects such as misalignments [1], improper torque in fastening [2], missing components [3], and surface imperfections [4]. Manual inspection, which is usually the conventional approach to QC, often fails in addressing all these issues, with practical limitations in inspection efficiency, operational costs, and potential for human error. QC processes can be improved through automated inspection using sensors, robotics, and computer vision (CV) systems, either individually or in combination. With use cases in a range of applications such as inspection and verification of assembly operations, production process control, and robot guidance, CV systems have proved to be effective in various tasks that are critical in industrial production, the most important of which include feature detection, recognition, segmentation, and 3D modeling [5]. Rule-based and learning-based algorithms are the most common classes, where depending on the choice, either a set of pre-programmed rules or supervised or unsupervised machine learning (ML) algorithms are used to detect features and patterns on the objects [6]. Quality inspection still requires human expertise due to complex defect classifications across products. This study proposes an automated solution combining robotics and a Deep Learning (DL) algorithm for real-time quality inspection on an assembly line. The method uses Convolutional Neural Networks (CNN) for defect detection and object placement verification. The system includes

2212-8271 © 2025 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the International Programme committee of the 58th CIRP Conference on Manufacturing Systems
10.1016/j.procir.2025.02.180

automated error correction by robots, creating a complete automated solution. This cost-effective approach using robotics and DL can benefit manufacturing plants, especially small and medium industries, by improving inspection efficiency and product quality. This study addresses two key challenges in industrial CNN deployment: the translation of DL models from development to edge computing environments, and the practical considerations of integrating these systems with existing assembly line infrastructure. The main contributions include: (1) empirical evidence of model performance trade-offs between development and deployment environments, (2) a framework for real-time defect detection integrated with robotic control systems, and (3) practical insights for deploying CNN-based inspection systems in resource-constrained and demanding industrial settings. These findings help bridge the gap between theoretical CNN capabilities and practical industrial implementation.

The remainder of this paper is organized as follows: Section 2 reviews relevant literature, Section 3 presents methodology and deployment framework, Section 4 describes the case study, Section 5 discusses results and analysis, Section 6 provides verification studies, and Section 7 concludes the paper.

2. Background

Various methods that deploy DL in defect detection applications in manufacturing have been surveyed in [7]. As the authors argue, compared with traditional defect detection methods such as ultrasonic or X-ray testing, DL-based methods generally have lower costs and provide better results. Another research has been done to study implementation of object detection techniques for detection of defects in steel sheet surfaces [8]. The object detection approach used in this research is proposed to be a model that utilizes a CNN as its backbone to extract characteristics of defects in different shapes. By comparing their model with the baseline model Faster R-CNN and its variations, the authors clock higher mean average precision (mAP), attributing it to better representation of multi-size defects as well as addition of the neural network to their model. In a separate study for a real-time application, YOLOX has been applied to detect safety and monitor production status on the shop floor of tobacco cabinets [9]. The production status is based on the feeding state which is itself measured in terms of deviation from the preset threshold, and this is done through obtaining the location information by YOLOX and then using Canny threshold method to determine whether the thresholds have been exceeded. The test results show higher accuracy compared with other similar methods. An example of leveraging DL to perform QC inspection over assembly lines is in [10] where authors collected data about product samples over a prototype assembly line with the goal of classifying the products into defective and non-defective. The context of application is limited to prototypes and the defects are created artificially to train the model. A DL-based study for detection of objects on the graphic cards assembly line has been conducted in [11] by combining two CNN models. While the foreign object detection in this study is based on Inception Resnet v2 model, the segmentation is performed through Attention Residual U-net++. The results showed superiority of the models in terms of performance in comparison with other methods in the study. Another approach to explore applications of DL on an assembly line can be seen in [12] where the authors compared three different models to investigate their performances in object detection by comparing the trade-off between accuracy and detection speed. As a result, YOLOv4 performed better than R-CNN and SSD models on a sample dataset of mechanical components. The study, however, focused only on the object detection task and did not include defect classification. In the study conducted in [13], the image features are extracted using 16 layers of CNN, producing a binary output for the failure state (defective or not), and a multiclass classification output for the type of defect. The authors propose Max pooling with rectified linear unit activation function to introduce non-linearity into the CNN. Using transfer learning to improve accuracy of the model which includes a pre-trained CNN to extract features from the images is also explored in this study. There are various studies that have explored deployment of DL and object detection for studying operations and cycle times on assembly lines. One example in this field is [14] where operators' repetitive actions have been studied in assembly settings through DL-based object detection. Furthermore, the Convolutional Pose Machine (CPM) is triggered after the tool is detected to estimate coordinates of human joint points which is in turn used to approximate operation time of the assembly actions. Results showed higher average accuracy of YOLOv3 compared with 2D CNN and 3D CNN in detection of objects. A similar study is [15] where step time and cycle time as well as operational anomalies are determined and identified continuously in each assembly cycle. This approach consists of an inference machine that contains a YOLOv7 model to detect assembly steps from a video stream and a "state machine" that times assembly steps for identification of any anomalies. A different method but similar objective is followed in [16] where the authors use a two-stage approach to monitor an operator's manual operation on the assembly line based on a YOLOv4-tiny model for feature extraction and then count by the pieces on the line through a sliding window algorithm. A comparison of this approach with other approaches demonstrates strong performance in key metrics such as mean absolute error and mAP.

3. Method and implementation

3.1. Model development

The overall framework used in this study for object detection is based on [17]. However, due to hyperparameters' significant impact on model performance, an optimization step is added to the current study. The process involves collecting and augmenting station images, applying data transformations, and using hyperparameter tuning to find optimal settings through iterations. The model is then trained, validated, and tested before deploying the best weights.

3.2. Optimization procedure

The hyperparameter optimization framework efficiently explores the search space through several key steps: defining an objective function that evaluates parameter sets, establishing the hyperparameter search space with specific data types and ranges, and creating a study with a sampler. The sampler suggests new hyperparameter combinations based on the search space and previous trial results. This iterative process continues until reaching specified stopping criteria (max trials, time limit, or convergence), ultimately returning the optimal parameter set. This is explained further in Section 4.

3.3. Model deployment

The model deployment framework operates through PLC-initiated commands that trigger the robot's assembly sequence. The camera captures frames which are processed by the trained algorithm for detection and classification. Based on verification results, the system either proceeds or initiates necessary retry procedures for detected defects. The flowchart for this process is depicted in Fig. 1.

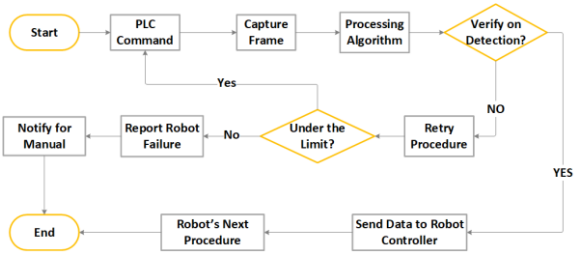

Fig. 1. Flowchart for real-time object detection and defect classification.

A critical point to consider in this process is the integration between PLC, the processing algorithm, and the robot controller. Before entering the cycle of robot's operation and camera verification, it is crucial to establish a connection to the algorithm to allow the system to receive external commands that might influence the process flow e.g., emergency stops or process updates. Furthermore, incorporating message handling throughout the process allows for dynamic adjustments based on real-time feedback from the algorithm, improving responsiveness and flexibility of the process. Finally, a feedback loop needs to be created from the camera's verification to the detection algorithm to transfer messages back and forth and adjust the robot's actions.

4. Case study

4.1. Station background

The study examines an automated chainsaw assembly line where industrial manipulators assemble various components. A specific robot station handles placement of snap-fit components (3 classes) and suspends (1 class) in different positions respectively across four chainsaw models, with positions varying due to different chassis dimensions. Previously, verification relied on a beam photoelectric sensor at the next station, which at the assembly point could not validate placement accuracy or allow re-attempts. To improve Overall Equipment Effectiveness (OEE), a new defect detection system is needed at the assembly station, though implementation is challenging due to similar colors and luminance between components and background. In addition, the station must introduce a closed-loop correction mechanism, allowing the robot to automatically attempt real-time defect corrections.

4.2. Model preparation and training

A set of 1225 images, all captured from the station with a 3024x4032 pixel dimension is initially annotated, with four object classes namely, SnapBL, SnapBR, SnapT, and Suspends. Afterwards, augmentations are applied to simulate potential variations in the station, increasing the total number of images to 11764, out of which, 980 images are used for validation, and 488 are designated as the test dataset. A study is conducted to optimize the hyperparameters. It is crucial to note that this operation is computationally intensive, therefore, the trials need to be defined within a reasonable search space. As for the hyperparameters setting, the list is narrowed down to a selection of: *Epochs, Image Size, Batch Size, Learning Rate,* and *Weight Decay*. The selection is mainly due to computation limitations and inclusion of potentially high impactful ones, otherwise the set can be extended to include additional hyperparameters related to data augmentation, optimizer settings, or model architecture. The hyperparameter search space used in this study is shown in Table 1.

Table 1. The search space used in the study for optimization of the hyperparameter set.

Trials	Epochs	Image size	Batch size	Learning rate	Weight decay	Optimizer	Model
5	50 – 100	416 – 640	8, 16	1e-4, 1e-3	1e-5, 1e-4	AdamW	Yolo V8.2s

The fitness function used for optimization is defined as a composite metric derived from the model's validation performance. Specifically, the objective function is defined as the negative of the model's fitness value, which is computed by YOLOv8's internal validation metrics combining precision, recall, and mAP:

$$Fitness = f\ (\mathrm{P} * \mathrm{R} * \mathrm{mAP50} * \mathrm{mAP50_95})$$

Where P = Precision, R = Recall, mAP50 = Mean Average Precision at 50% IoU threshold, and mAP50_95 = Mean Average Precision averaged over IoU thresholds from 50%-95%.

Based on the outputs of the optimizer, the results for each trial are obtained which can be seen in Table 2.

Table 2. The output of optimization for different hyperparameters and the value of objective function according to those values.

Nr.	Epochs	Image size	Batch size	Learning rate	Weight decay	Negative fitness(-)

1	89	416	16	0.00047	4.79e-05	0.67249
2	61	640	16	0.00074	1.17e-05	0.68225
3	73	416	16	0.00096	8.52e-05	0.66441
4	77	416	8	0.00060	2.76e-05	0.66407
5	69	416	16	0.00023	1.92e-05	0.67389

As observed from the results, the second trial outputs the best set of hyperparameters. These values are then fed into the settings of the model (yolov8s) for training over the dataset. The training results can be seen in Table 3. The model demonstrates efficient processing capabilities with a total processing time of 6.4ms per image, broken down into pre-processing (0.4ms), inference (4.9ms), and post-processing (1.1ms). This enables processing of approximately 156 frames per second, making it suitable for high-speed assembly line operations where real-time detection is critical.

Table 3. The optimized model's performance metrics in individual classes and overall, on the dataset of 980 images.

Class	Instances	Box (P	R	mAP50	mAP50-95)
All	847	0.962	0.967	0.972	0.647
SnapBL	167	0.977	0.976	0.984	0.711
SnapBR	104	0.989	1	0.995	0.673
SnapT	262	0.977	0.962	0.979	0.764
Suspends	314	0.906	0.93	0.932	0.441

4.3. Model deployment on the station

The system uses an ABB robot programmed in RAPID, connected to a Python detection application through a server-client architecture. The main procedure establishes connection between the robot controller (server) and Python application (client), binding to a local IP address. The communication cycle operates as follows:

- The robot controller requests object detection from the client
- The client returns detection data (number of objects detected)
- For each detected object, the robot receives: Class IDs, object coordinates, and detection confidence scores

The client then sends summary data including total detections and missed objects. Based on this information, the robot plans and executes movements, including corrective actions for missed objects or re-execution of operations. This cycle continues for subsequent frames, enabling continuous monitoring and verification of assembly operations.

5. Results and analysis

5.1. Confusion matrix

The elements in the normalized confusion matrix in Fig. 2 show the model's performance for each class. The diagonal elements (SnapBL: 0.98, SnapBR: 1.0, SnapT: 0.98, Suspends: 0.96) indicate high accuracy for these classes, with the SnapBR class achieving perfect accuracy. The off-diagonal elements are relatively low, suggesting minimal confusion between classes. However, there is some confusion between the Suspends class and the background (0.07), indicating potential false positives or negatives for this class. Overall, the confusion matrix demonstrates a promising performance for most classes, with the Suspends class being slightly more challenging than the rest.

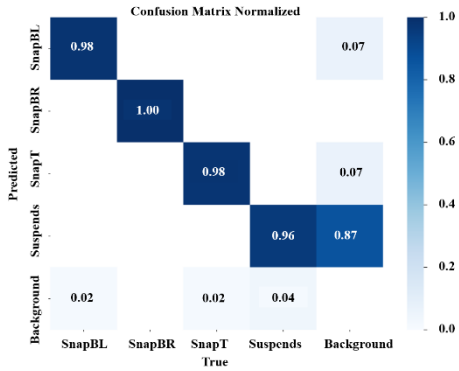

Fig. 2. The normalized confusion matrix in the optimized model.

5.2. Precision-recall curve

According to the Precision-Recall (PR) curve in Fig. 3, the SnapBR class achieves the highest average precision (AP) of 0.995, indicating strong performance in detecting this class. The SnapBL and SnapT classes also have high APs of 0.984 and 0.979, respectively, demonstrating compelling performance. The Suspends class has a lower AP of 0.932, which aligns with the slightly weaker performance observed previously. The mean average precision at intersection over union (IoU) of 0.5 (mAP@0.5) is 0.972, which is considered a good result. The PR curve confirms high precision and recall achieved for most classes, with the Suspends class being the most challenging but still performing reasonably well.

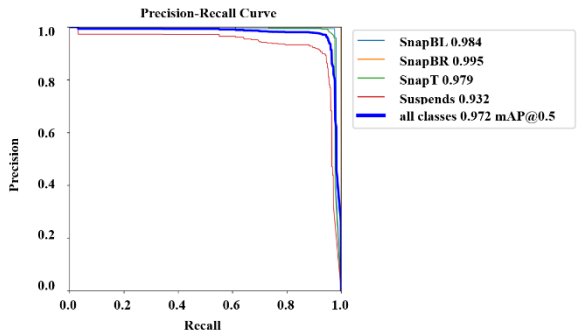

Fig. 3. The precision-recall curve in the optimized model.

5.3. F1-confidence curve

The F1-Confidence curve shown in Fig. 4 visualizes the trade-off between confidence and the F1 score, which combines precision and recall. All classes exhibit a similar curve shape, with F1 scores initially increasing rapidly with confidence and then plateauing or declining at higher confidence levels. The SnapBL, SnapBR, and SnapT classes achieve high F1 scores (around 0.96) at a confidence threshold of 0.458, indicating good overall performance. The Suspends class has a slightly lower F1 score at the same

confidence threshold, suggesting potential room for improvement. The F1-Confidence curve confirms strong performance of the model across classes, with the Suspends class being slightly weaker but still achieving reasonably high F1 scores.

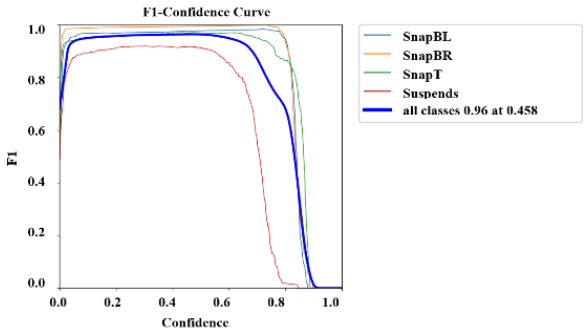

Fig. 4. The F1-confidence curve in the optimized model.

5.4. Final results and overall performance

The results plot in Fig. 5 provides insights into the model's training behavior and convergence. The box loss, class loss, and DFL (dense feature loss) all exhibit smooth convergence behavior, decreasing steadily throughout the training process. The precision (B) and recall (B) curves demonstrate stable performance, with precision reaching around 0.96 and recall around 0.95 towards the end of training. The mAP50 (B) and mAP50-95 (B) curves, show steady improvement during training, reaching values around 0.96 and 0.65 respectively. The overall result indicates that the model converged well, with stable and improving performance metrics throughout the training process.

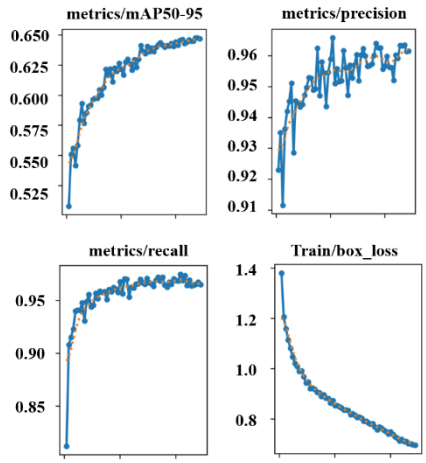

Fig. 5. The metrics for training box loss, mAP50-95, recall, and precision in the optimized model.

Based on the provided metrics, the trained YOLOv8 model demonstrates high performance for the object detection task on the given dataset. The high accuracy values in the confusion matrix, coupled with the strong precision, recall, and mAP scores, indicate that the model can reliably detect and classify the SnapBL, SnapBR, and SnapT classes with high accuracy. The Suspends class, while slightly weaker than the others, still achieves reasonable performance, with an AP of 0.932 and mAP@0.5 of 0.972. The smooth convergence behavior during training and the steady improvement in performance metrics suggest that the model's training is without significant issues such as overfitting or divergence.

6. Verification

To verify results of the optimization, another model with fixed settings for the hyperparameters is created. To compare the results from the previous step (called as optimized model) with a reference model where the above considerations are considered (called as verification model), a study is designed to include the hyperparameter settings in Table 4.

Table 4. The hyperparameter settings used in the verification model.

Epochs	Image size	Batch size	Learning rate	Weight decay	Warm up epochs	Warm up momentum	Warm up bias	Optimizer	Model
200	800	50	0.001	0.0005	5	0.5	0.01	AdamW	YoloV8.2m

6.1. Comparative analysis

The YOLOv8m verification model and YOLOv8s optimized model exhibit significant architectural differences in their configurations. The verification model's configuration is chosen to represent a realistic industrial deployment scenario, such as migrating from development servers to edge computing devices in production lines, where multiple parameters need simultaneous adjustment to meet hardware constraints, line speed requirements, and quality control specifications. Rather than isolating individual parameter effects, this comparison focuses on overall system performance in such industrial settings. The verification model runs for 200 epochs compared to the optimized model's 61 epochs, utilizing a larger input image size of 800 versus 640, and a higher batch size of 40 compared to 16. Training is conducted on an NVIDIA Jetson Orin AGX with 64 GB RAM, requiring over 45 hours to complete. The verification model employs a higher learning rate of 0.01 (versus 0.000749), complemented by stability-enhancing features including 3 warmup epochs, 0.8 warmup momentum, and 0.01 warmup bias learning rate. While these parameters potentially offer better generalization, they demand substantially more computational resources. The warmup parameters help stabilize the training process by enabling gradual parameter adjustments in the early stages. While the verification model is not subjected to hyperparameter optimization, it still achieves better performance. This suggests that the model's configuration is robust enough for industrial deployment even without extensive tuning. This robustness is particularly valuable in production environments where extensive hyperparameter optimization may not be feasible due to time and resource constraints.

6.2. Crosschecking against optimized results

The verification model demonstrates different training characteristics due to its modified configuration. The loss and metric curves show smooth convergence behavior despite the changes in multiple parameters including epochs, learning rate, and warmup settings. The model's final performance metrics show marginal improvements in precision, recall, and mAP values compared to the optimized model. However, these improvements come with significant trade-offs in computational requirements and longer training times i.e., clocking +46 hours on a 50W Jetson 64GB RAM for the verification model against +14 hours on an NVIDIA RTX 3080 32GB RAM for the optimized model. Looking at the metrics in Table 5 and comparing them with those found in Table 3, it can be observed that there are visible improvements in key metrics, particularly in mAP50-95 of classes which shows an improvement of 2.6 – 3.8%. The difference in the fitness value also shows slight improvement i.e., for the optimized model it is ≈ 0.679 while for the verification model it is ≈ 0.703.

Table 5. The verification model's performance metrics in individual classes and overall, on the dataset of 980 images.

Class	Instances	Box (P	R	mAP50	mAP50-95)
All	847	0.961	0.974	0.979	0.673
SnapBL	167	0.965	0.982	0.989	0.731
SnapBR	104	0.987	1	0.995	0.711
SnapT	262	0.981	0.977	0.985	0.793
Suspends	314	0.911	0.936	0.947	0.459

The intention behind the use of different hardware configurations is to validate deployment feasibility across industrial settings. While initial training and optimization are performed on a laptop GPU, verification on an edge device demonstrates the model's robustness for industrial deployment where edge computing is preferred due to latency and data privacy requirements. The consistent performance across platforms (2.6-3.8% difference) validates the model's adaptability to different computational resources typical in manufacturing environments.

7. Conclusion

This study presents a DL-based visual inspection system for assembly line quality control with integrated robotic process control. The system processes camera feeds through CNN models, controlling the robot's next action or initiating retry procedures, with operator notification as a fallback. Comparing an optimized model against a verification model with larger parameters showed marginal performance improvements (2.6-3.8%) but required significantly more computational resources and training time. The optimized model proves sufficient for practical deployment, with model selection depending on available computational resources and application requirements.

Acknowledgement

The authors acknowledge Sweden's Innovation Agency Vinnova and KK-Stiftelsen for supporting this research.

References

1. Dalle Mura, M., Dini, G.: An augmented reality approach for supporting panel alignment in car body assembly. J. Manuf. Syst. 59, 251–260 (2021). https://doi.org/10.1016/J.JMSY.2021.03.004
2. Rusli, L., Luscher, A.: Fastener identification and assembly verification via machine vision. Assem. Autom. 38, 1–9 (2018). https://doi.org/10.1108/AA-08-2016-093/FULL/PDF
3. Killing, J., Surgenor, B.W., Mechefske, C.K.: A machine vision system for the detection of missing fasteners on steel stampings. Int. J. Adv. Manuf. Technol. 41, 808–819 (2009). https://doi.org/10.1007/S00170-008-1516-3/METRICS
4. Ren, Z., Fang, F., Yan, N., Wu, Y.: State of the Art in Defect Detection Based on Machine Vision. Int. J. Precis. Eng. Manuf. Technol. 2021 92. 9, 661–691 (2021). https://doi.org/10.1007/S40684-021-00343-6
5. Zhou, L., Zhang, L., Konz, N.: Computer Vision Techniques in Manufacturing. IEEE Trans. Syst. Man, Cybern. Syst. 53, 105–117 (2023). https://doi.org/10.1109/TSMC.2022.3166397
6. Ebayyeh, A.A.R.M.A., Mousavi, A.: A Review and Analysis of Automatic Optical Inspection and Quality Monitoring Methods in Electronics Industry. IEEE Access. 8, 183192–183271 (2020). https://doi.org/10.1109/ACCESS.2020.3029127
7. Yang, J., Li, S., Wang, Z., Dong, H., Wang, J., Tang, S.: Using deep learning to detect defects in manufacturing: A comprehensive survey and current challenges, https://www.mdpi.com/1996-1944/13/24/5755, (2020)
8. Hao, R., Lu, B., Cheng, Y., Li, X., Huang, B.: A steel surface defect inspection approach towards smart industrial monitoring. J. Intell. Manuf. 32, 1833–1843 (2021). https://doi.org/10.1007/s10845-020-01670-2
9. Wang, C., Zhao, J., Yu, Z., Xie, S., Ji, X., Wan, Z.: Real-Time Foreign Object and Production Status Detection of Tobacco Cabinets Based on Deep Learning. Appl. Sci. 12, 10347 (2022). https://doi.org/10.3390/app122010347
10. Mandapaka, S., Diaz, C., Irisson, H., Akundi, A., Lopez, V., Timmer, D.: Application of Automated Quality Control in Smart Factories - A Deep Learning-based Approach. SysCon 2023 - 17th Annu. IEEE Int. Syst. Conf. Proc. (2023). https://doi.org/10.1109/SYSCON53073.2023.10131100
11. Kuo, R.J., Nursyahid, F.F.: Foreign objects detection using deep learning techniques for graphic card assembly line. J. Intell. Manuf. 34, 2989–3000 (2022). https://doi.org/10.1007/S10845-022-01980-7/TABLES/10
12. Sureshkumar, S., Mathan, G.E., Ri, P., Govindarajan, M.: Deep learning framework for component identification. Int. J. Inf. Technol. 14, 3301–3309 (2022). https://doi.org/10.1007/s41870-022-00895-z
13. Kovilpillai, J.J.A., Jayanthy, S.: An optimized deep learning approach to detect and classify defective tiles in production line for efficient industrial quality control. Neural Comput. Appl. 35, 11089–11108 (2023). https://doi.org/10.1007/s00521-023-08283-9
14. Chen, C., Wang, T., Li, D., Hong, J.: Repetitive assembly action recognition based on object detection and pose estimation. J. Manuf. Syst. 55, 325–333 (2020). https://doi.org/10.1016/j.jmsy.2020.04.018
15. Selvaraj, V., Al-Amin, M., Tao, W., Min, S.: Intelligent assembly operations monitoring with the ability to detect non-value-added activities as out-of-distribution (OOD) instances. CIRP Ann. 72, 413–416 (2023). https://doi.org/10.1016/J.CIRP.2023.04.027
16. Lou, P., Li, J., Zeng, Y.H., Chen, B., Zhang, X.: Real-time monitoring for manual operations with machine vision in smart manufacturing. J. Manuf. Syst. 65, 709–719 (2022). https://doi.org/10.1016/J.JMSY.2022.10.015
17. Ashourpour, M., Azizpour, G., Johansen, K.: Real-Time Defect and Object Detection in Assembly Line: A Case for In-Line Quality Inspection. In: Lecture Notes in Mechanical Engineering. pp. 99–106 (2024)